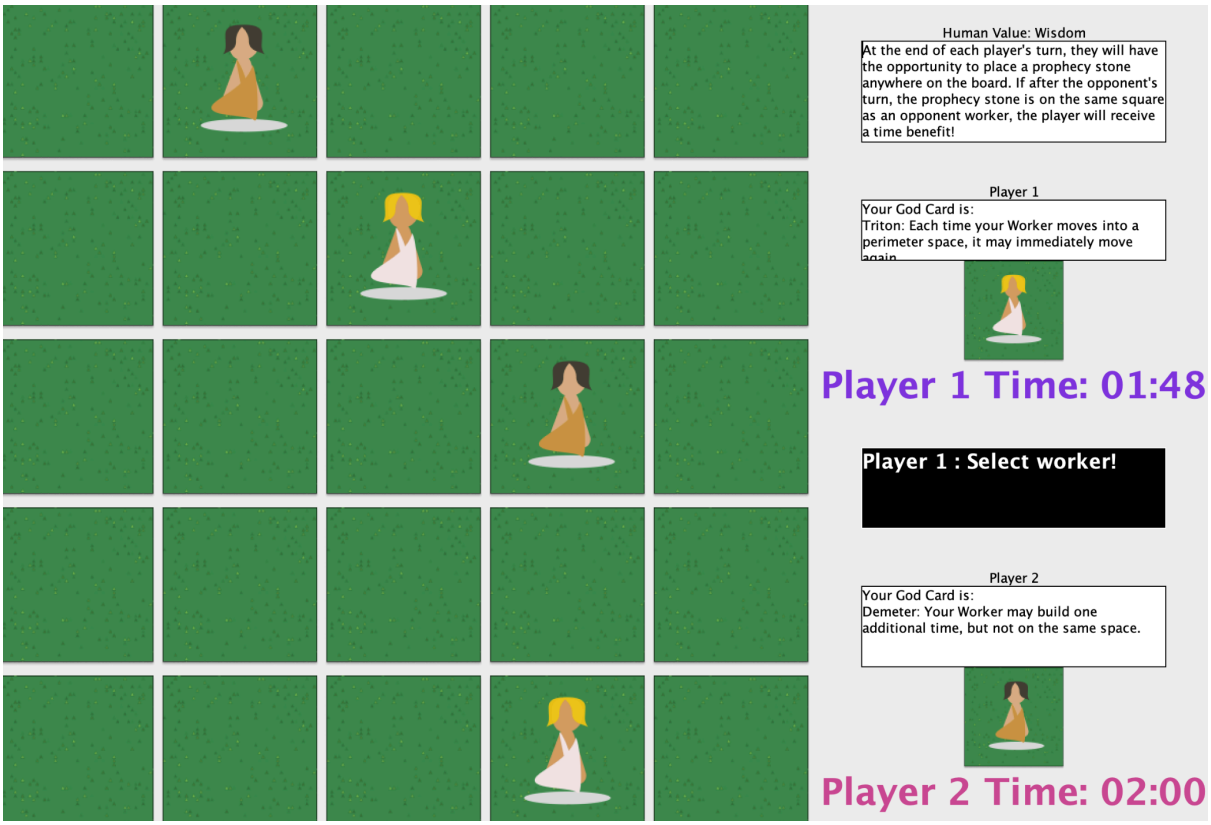
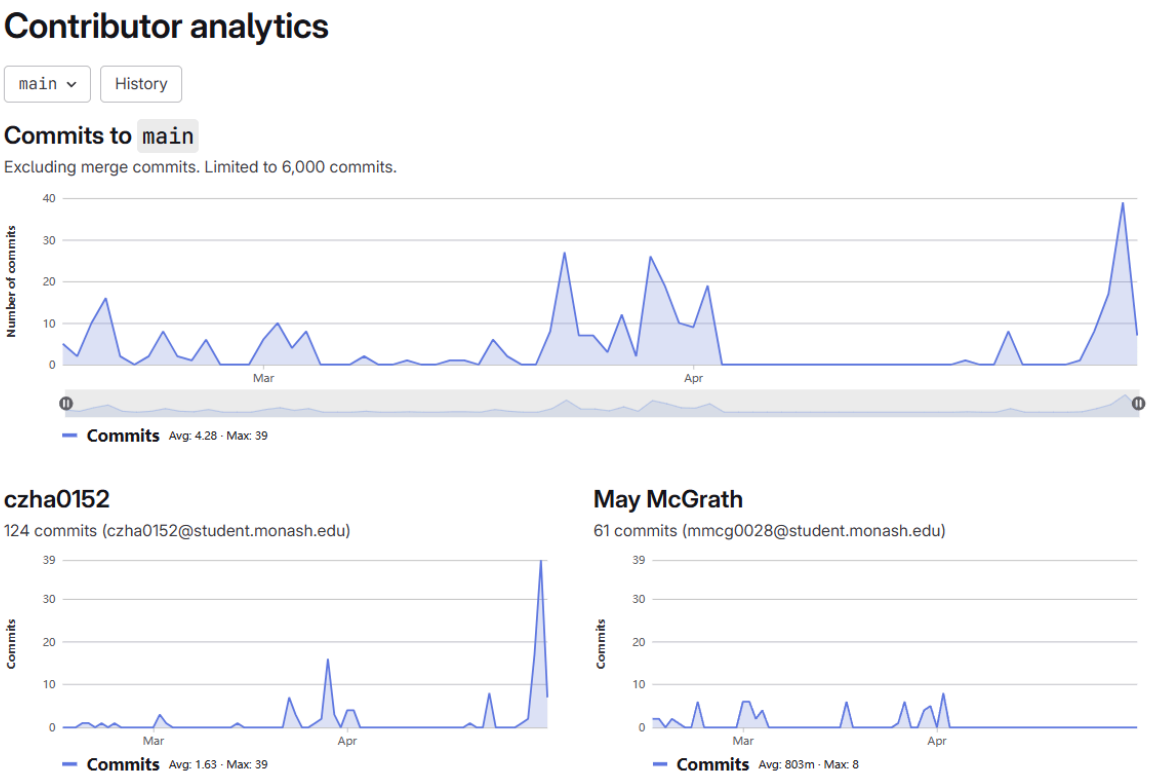
FIT3077 Sprint 3 - Claire Zhang

(Student *ID:* 33111332)



Contributor Analytics



Description of Extensions

1. Required: Triton GodCard

Each time your Worker moves into a perimeter space, it may immediately move again

2. Pick-From-The-List: Timer

Each player gets a fixed amount of time (2 minutes, but can be reconfigured in TimedPlayerDecorator) for “thinking” and executing their turns. Once a player has completed their turn, their clock pauses and the clock starts for the next player. If the clock of a player reaches zero, this player loses the game. The timers for each player are made visible in the user
interface so that each player can see how much time they have left.

3. Self-Defined: Prophecy Stones

At the end of each player's turn, they will have the opportunity to place a prophecy stone anywhere on the board. If after the opponent's turn, the prophecy stone is on the same square as an opponent worker, the player will receive a time benefit!

Updated class diagram

See next page

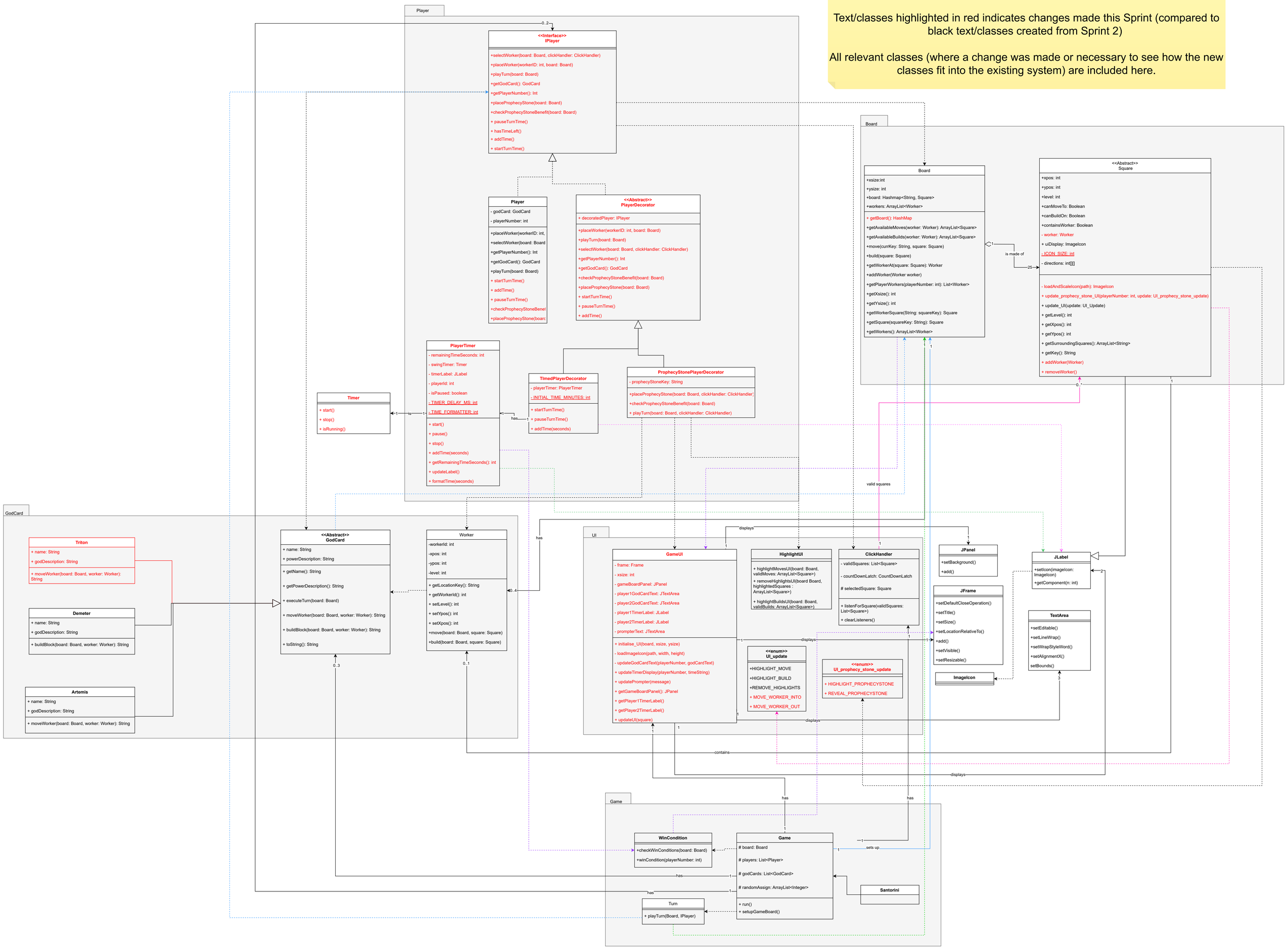
Description: The UML has been updated to reflect changes made in Sprint 3. These changes are highlighted in red. Other colours are used to make the diagram more readable where lines cross over one another. It does not include classes where no changes were made, such as subclasses of Square.

Significant changes include adding the Player package, where IPlayer, PlayerTimer, PlayerDecorator and concrete decorators have been added. Additionally, GameUI has been added to UI package to setup and manage the game board UI, instead of Board. In Sprint 3, Triton extends GodCard just as other God cards, Artemis and Demeter, were implemented in Sprint 2.

UML for timer, god card and human value (wisdom: prophecy stone) extensions.

Text/classes highlighted in red indicates changes made this Sprint (compared to black text/classes created from Sprint 2)

All relevant classes (where a change was made or necessary to see how the new classes fit into the existing system) are included here.



Three CRC cards

See next 3 pages

Description - TimedPlayerDecorator: This card details the TimedPlayerDecorator's role in extending an IPlayer object. Its primary responsibility is to manage a player's countdown timer, which involves starting, pausing, reporting, and adding time to the timer, while also delegating core player actions to the wrapped IPlayer.

Description - ProphecyStonePlayerDecorator: This card outlines the ProphecyStonePlayerDecorator, which wraps an IPlayer to manage prophecy stone functionalities. It is responsible for storing placed prophecy stone locations, guiding players through the placement process, revealing stones, checking if a revealed stone's square has an opponent, and awarding a time benefit if the prophecy is fulfilled, all while delegating core player actions.

Description - PlayerTimer: The Player Timer card specifies a component responsible for managing a player's remaining time. It handles starting, pausing, and adding time to the countdown, primarily interacting with javax.swing.Timer for these operations.

CRC Cards

TimedPlayerDecorator	
Decorate an IPlayer	IPlayer
Manage a player's countdown timer	PlayerTimer
Start the timer at turn start	PlayerTimer, Turn, Game
Pause timer at turn end	PlayerTimer, Turn, Game
Report remaining time	PlayerTimer, GameUI
Add time to timer	PlayerTimer
Delegate core player actions	IPlayer

CRC Cards

Prophecy Stone Player Decorator	
Decorate an IPlayer	IPlayer
Store the player's placed prophecy stone location	Square
Guide player to place prophecy stone	GameUI, square, ClickHandler
Reveal a placed stone	square, GameUI
Check if revealed stone's square has opponent	Square
Award time benefit if prophecy fulfilled	IPlayer
Delegate core player actions	IPlayer

CRC Cards

Player Timer	
Manage remaining time for a Player	jaxax.swing.Timer
Start countdown	jaxax.swing.Timer
Pause countdown	jaxax.swing.Timer
Add more time to remaining time	jaxax.swing.Timer
Format and update a UI label with current time	JLabel
Trigger Win when player's timer runs out	WinCondition

Design Rationales

IPlayer Interface:

The IPlayer interface was introduced to establish a consistent contract for all player-related objects, including Player and its decorators. This ensures that both the base player and extended player functionalities adhere to a common set of methods, fostering polymorphism and loose coupling while specifically defining the blueprint for player behavior rather than providing an implementation, and it did not involve reassigning previous responsibilities.

PlayerDecorator Abstract Class:

As an abstract class implementing IPlayer, PlayerDecorator serves as the foundational element for concrete decorators, holding a reference to an IPlayer object. Its purpose is to enable the dynamic addition of features to a Player by wrapping, rather than directly modifying, the core Player class, thus ensuring extensibility without violating design principles, and it represents a new architectural component rather than a reassignment.

TimedPlayerDecorator Concrete Decorator:

This concrete decorator extends PlayerDecorator to augment a player's behaviour with time-limited turns by managing a countdown timer. This allows for flexible integration of timer functionality as an optional, runtime-addable feature, preventing the Player class from becoming a "god class" burdened with excessive responsibilities, and it implicitly shifted the responsibility of timer management to a composable component.

ProphecyStonePlayerDecorator Concrete Decorator:

ProphecyStonePlayerDecorator extends PlayerDecorator to incorporate prophecy stone mechanics, allowing for the strategic placement, revelation, and benefit application of stones. This approach seamlessly integrates the feature into the game without modifying the fundamental Player class, maintaining a lean and flexible design by ensuring features can be added independently, and it did not involve reassigning prior responsibilities.

GameUI Class:

The GameUI class was established to centralise the graphical user interface's setup and updates, including initialising the board display, updating timer labels, and prompter messages. This proactive refactoring separated UI concerns from the Board class, which previously handled both game state and presentation, thereby making Board more focused on its core domain logic and preventing it from becoming a "god class" with too many responsibilities.

Alternative 1: Extending the Player class with timer and stone attributes:

- **Description:** Initially, I considered extending the Player class directly by adding attributes like timer and stone, along with corresponding methods such as addStone() and removeStone(). (This is referenced in s3_uml_1_timer_and_stone).
- **Reason for not using:** While this design was simple and intuitive for a limited number of features, it proved to be inflexible and did not support extensibility. If more features were to be added to Player beyond the two (timer and prophecy stone), the Player class could quickly become a "god class," managing the timer, stone, and potentially many other functionalities all at once. This would lead to tight coupling, making the class difficult to maintain, test, and modify in the future without affecting other parts of the system.

Alternative 2: ProphecyStone as a separate class:

- **Description:** I also thought about having ProphecyStone as a completely separate class.
- **Reason for not using:** This seemed like an extra unnecessary step. All the information needed to store a prophecy stone was its location on the board, and the primary interaction involved updating the UI to reveal or remove the stone. Creating a class for this simple data and UI interaction felt like over-engineering, adding complexity without significant benefits compared to handling its logic within the player's context (ProphecyStonePlayerDecorator).

Decorator Design Pattern:

- **Why and where applied:** The Decorator design pattern was used for the timer and prophecy stone extensions. This pattern was applied because it allows for extending a class's functionality by wrapping its objects, leaving the base class unmodified. This ability was highly useful for both features, as it allowed me to extend the Player class without adding new attributes and methods directly into Player. Instead, I could "decorate" it with TimedPlayerDecorator and ProphecyStonePlayerDecorator.
- **Benefits:** This provides a more flexible and extensible design. For example, if the timer and prophecy stones were to be optional features or part of specific game modes, they could be added or removed from the Player object at runtime. This dynamic extensibility would not be simple to implement if these features existed directly inside the Player class.
- **Further Justification:** While IPlayer and its implementers (Player, PlayerDecorator, TimedPlayerDecorator, ProphecyStonePlayerDecorator) might appear extensive due to the number of methods, this is necessary and valid in this context. The IPlayer interface must define all the core actions a player can perform in the game, as well as any actions that can be augmented by decorators. This comprehensive interface ensures that decorators can transparently wrap and enhance the base player's behaviour, adhering to the Liskov Substitution Principle. The extensiveness reflects the rich set of interactions a player has with the game board and other game elements, and each method serves a distinct purpose in the game logic.

Extension Idea: Prophecy Stones

Claimed Human Value: Wisdom (Universalism category)

Why Wisdom is Important: Wisdom, as a human value, is crucial because it encompasses foresight, the ability to learn from past experiences, and the capacity to apply knowledge and understanding to make sound judgments. It encourages thoughtful decision-making and strategic thinking, which are valuable traits in both games and real-life scenarios.

How Wisdom is Manifested in the Game:

- Prophecy stones are a feature designed to encourage foresight and the ability to learn from past actions, directly reflecting the insightful nature of wisdom.
- **Functionality:** After each player completes their turn, they secretly place one Prophecy Stone on a space on the board. At the beginning of their next turn, they can reveal a stone they placed. If the space where the stone was placed is now occupied by an opponent's worker, the player gains a small benefit of an additional 30 seconds to their timer.
- **Player Experience:** This functionality directly encourages players to anticipate their opponent's moves and strategically place the stones. To effectively utilise this advantage, players must develop a keen understanding and knowledge of their opponent's playstyle. For instance, by observing a strategic opponent's consistent patterns of movement and building, a player can predict their next move and place a prophecy stone accordingly. This process of learning from previous moves to predict future actions is a core aspect of wisdom.
- **Broader Impact (Broadmindedness):** The mechanic also subtly encourages a more varied and exciting gameplay. If a player wants to prevent their opponent from gaining the timer benefit, they might consciously vary their own gameplay, moving their workers to unpredictable squares or choosing workers that the opponent would not suspect. This requires a different kind of foresight and adaptive thinking, which touches on "broadmindedness," another Universalism human value.

Changes in the Game to Enable This:

- Implemented the ProphecyStonePlayerDecorator to add the prophecy stone capabilities to the Player class.
- Added logic within the turn structure to allow players to place a prophecy stone at the end of their turn.
- Implemented logic at the beginning of a player's turn to allow them to reveal a previously placed stone and check for the opponent's worker presence on that square.
- Integrated the mechanism to grant the TIME_BENEFIT (30 seconds) to the player's timer via decoratedPlayer.addTime(TIME_BENEFIT) if the prophecy condition is met.
- Added UI elements or updates with GameUI.updatePrompter messages, visual indicators for prophecy stones on the board to communicate the placement, revelation, and outcome of prophecy stones to the players.

Description of Executable

Instructions to create executable of Santorini (Sprint 3.jar) In Intellij IDEA IDE on MacOS:

1. Mark "src" directory as Sources root, and “images” directory as Resources root
2. In the toolbar, select File -> Project Structure... -> Artifacts
3. Click "+" and select Add Jar -> from modules with dependencies...
4. Select "Santorini" as Main Class and click "OK"
5. Input the output directory you want, e.g. for me:
"/Users/chickenbum/Documents/FIT3077/Sprint 3/out/artifacts/Sprint_3_jar" and click “OK”
7. In the toolbar, select Build -> Build Artifacts... -> Sprint 3:jar -> Build
Done!

Running the executable:

Run the executable by simply double-clicking it in Finder. Alternatively, in terminal, cd into the output directory, e.g. for me:
“/Users/chickenbum/Documents/FIT3077/Sprint 3/out/artifacts/Sprint_3_jar” and enter the following command:

```
java -jar Sprint\ 3.jar
```

Requirements:

Ensure you have the following installed:
JDK version: Eclipse Temurin JDK 24 (aarch64)
Architecture: ARM64 (Apple Silicon)
JRE Version (for running): Java Runtime Environment 24

Reflection on Sprint 2 Design

Overall Difficulty of Incorporating Extensions:

Triton (Required Extension): Easy!

- **Aspects of Sprint 2 Design:** The implementation of Triton was straightforward due to the pre-existing Template Method design pattern for GodCards.
- **Reason for Ease:** I only had to create a method in Board to check if a square is a perimeter square, adding the concrete Triton GodCard class, and overriding moveWorker() in Triton to call its parent class's moveWorker() to perform the first default move, then creating a while loop that coordinates the click handling, UI updates and move logic if the worker is within the perimeter. This confirmed that our template method design pattern was a good choice, as it saved us from duplicating existing moveWorker() logic and buildBlock() code.

Human Value & Timer (Prophecy Stones and Player Timers): Harder.

- **Aspects of Sprint 2 Design:** The difficulty stemmed primarily from how UI elements were handled in Sprint 2, specifically the tight coupling between UI and game logic within the Board class. This class had too many responsibilities, acting as a "god class" that managed game state, logic, and its graphical representation.
- **Reason for Difficulty:** Integrating new visual features for player timers and prophecy stone indicators became cumbersome due to this tight coupling. Introducing new UI components without significant complexity or potential errors was challenging. The process was further complicated by the need to manually read uncommented legacy code line by line to understand existing logic, and the repeated creation of HighlightUI instances.
- **Solution for Difficulty:** This necessitated a significant refactoring effort. As per Sprint 2 feedback, I separated the Board UI from the game logic by creating a dedicated GameUI class, which now initialises and manages the board's visual elements. This keeps the Board class more focused on maintaining game state, leading to a cleaner, more manageable, and robust design.

  To further streamline the UI, I refactored the image loading and scaling process in Square into a reusable loadAndScaleIcon helper method. I also modified the Square class to explicitly store the Worker object present on it, which greatly simplified retrieving the worker at a square, which was previously inefficiently done by iterating through all workers stored in the Board list and matching their location key against the square’s key. This also optimised Square loading the correct worker at a square when the UI is updated (highlight removed or when worker moves into the square), which requires the worker’s id.

  Unused code was removed, and HighlightUI methods were converted into static methods, eliminating the need to create local instances and making the code cleaner.

Changes in Sprint 2

If I were to go back to Sprint 2 and could start all over again, I would have strongly advocated for implementing the Model-View-Controller (MVC) design pattern from the outset. This architectural pattern would have separated game logic (Model), UI presentation (View), and user input handling (Controller) more distinctly. This clear separation of concerns would have made it significantly easier to see who is responsible for what, leading to a more modular and extensible codebase from the very beginning. The UI-related challenges encountered in Sprint 3 would have been greatly mitigated by an initial MVC adoption.

AI Acknowledgement

I used Gemini 2.5 Flash (https://gemini.google.com/app) to generate ideas for the human value extension by inputting the brief and example extensions given. The prophecy stone wisdom was chosen from a list of a few ideas it provided and modified by me to tie the benefit to the other extension I chose (Timer). Gemini was also used to provide pros and cons of implementing the timer and prophecy stone extension with Decorator design pattern versus my previous design, aiding in my design thinking and justification.

I used ChatGPT 4o (https://chatgpt.com/) to generate the player1_prophecystone.png, player1_prophecystone_highlight.png, player2_prophecystone.png and player2_prophecystone_highlight.png images.